



UNIVERSIDADE KIMPA VITA
INSTITUTO POLITÉCNICO
CURSO DE ENGENHARIA INFORMÁTICA

RELATÓRIO DE LINGUAGEM DE PROGRAMAÇÃO VI-SISTEMA HOSPITALAR

Classe: 3º Ano

Turma: TL 101

Período: Manhã

Grupo:

Integrantes

Augusto Paulo

Cardoso Faniquisa

Fernando Lourenco A.Jorge

Manuel Antonio Tolentino

Makuiza Eduardo

O DOCENTE

Moyo Kanivengidio

UÍGE /2026

Índice

1. INTRODUÇÃO E ENQUADRAMENTO DO PROBLEMA.....	3
2. REQUISITOS E ARQUITETURA DO SISTEMA.....	4
3. MODELAÇÃO DE DADOS (DIAGRAMA DE CLASSES UML).....	4
4. O MÓDULO DE INTELIGÊNCIA ARTIFICIAL.....	5
5. INTERFACE GRÁFICA DO UTILIZADOR (GUI) E DESIGN SYSTEM... 	6
6. CONCLUSÃO.....	8
7. REFERÊNCIAS BIBLIOGRÁFICAS.....	9
ANEXO: GUIA DE EXECUÇÃO E TESTE RÁPIDO.....	10

1. INTRODUÇÃO E ENQUADRAMENTO DO PROBLEMA

A superlotação nos serviços de urgência hospitalar constitui um dos maiores desafios da gestão de saúde pública atual. A demora na classificação inicial do risco clínico pode resultar no agravamento de quadros agudos ou, em casos extremos, na perda de vidas humanas. O método tradicional de triagem baseia-se na avaliação puramente manual de protocolos clínicos, um processo suscetível a erros humanos derivados da fadiga ou da subjetividade interpretativa.

O presente projeto propõe uma solução tecnológica para mitigar esta problemática através do desenvolvimento de um **Sistema Hospitalar Inteligente**. O objetivo principal é automatizar o processo de triagem de pacientes em tempo real, utilizando um modelo preditivo de Aprendizagem Automática (*Machine Learning*). O sistema analisa os sinais vitais recolhidos na receção e estabelece instantaneamente uma Fila de Atendimento Otimizada por gravidade clínica, garantindo a priorização científica dos cuidados de saúde.

2. REQUISITOS E ARQUITETURA DO SISTEMA

Para garantir a escalabilidade, segurança e de acordo com as boas práticas de desenvolvimento, o ecossistema de software foi projetado de forma modular, separando a persistência de dados, a lógica de negócio e os componentes de apresentação visual.

O projeto adota uma arquitetura inspirada no padrão **MVC (Model-View-Controller)**, eliminando o acoplamento rígido através da seguinte árvore de diretórios:

- **database/ (Camada de Dados):** Contém o arquivo `conexao.py` responsável por estabelecer a sessão local com o motor SQLite e o arquivo `modelos.py` que define o mapeamento objeto-relacional (ORM) via SQLAlchemy.
- **core/ (Camada de Serviços):** Centraliza as regras de negócio em `servicos.py`. Trata da encriptação de dados sensíveis e das queries avançadas de ordenação.
- **ai/ (Camada de Inteligência Artificial):** Contém o pipeline de treino (`modelo_triagem.py`) e o arquivo binário persistido (`modelo_triagem.pkl`) com a floresta de decisão guardada.
- **gui/ (Camada de Apresentação):** Implementa as interfaces gráficas em CustomTkinter (`login.py` e `dashboard.py`).

3. MODELAÇÃO DE DADOS (DIAGRAMA DE CLASSES UML)

A modelação do banco de dados relacional foi estruturada para garantir a integridade referencial e evitar anomalias de inserção ou redundância de dados.

Abaixo encontra-se a especificação do **Diagrama de Classes UML**, mapeando as entidades estruturadas na camada de persistência:

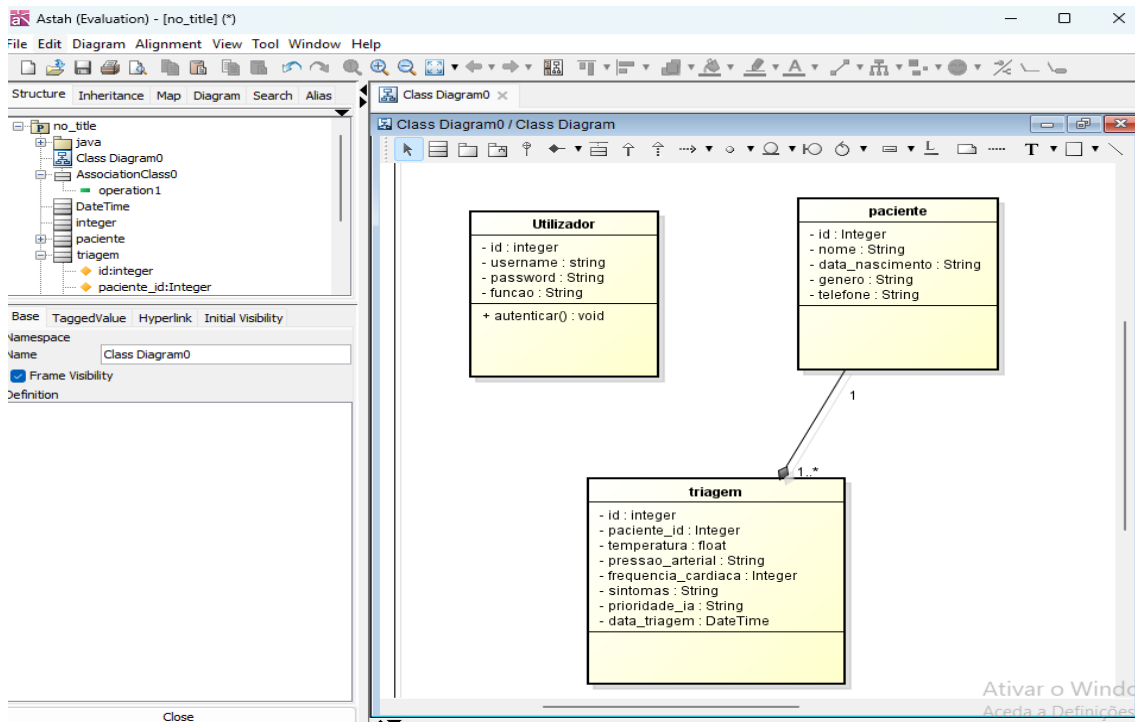


Diagrama UML

Análise de Relacionamentos e Isolamento de Funções:

1. **Classe Utilizador:** Atua de forma independente no modelo. O seu propósito exclusivo é o controlo de acessos e segurança perimetral (Autenticação). Não possui chaves estrangeiras ligadas ao fluxo clínico direto para garantir a auditoria limpa do sistema.
2. **Relação Paciente-Triagem:** Configura uma associação de **1 para Muitos (1..*)**. Um único paciente mapeado pode possuir múltiplos registos de triagem ao longo do seu histórico clínico hospitalar, permitindo o acompanhamento longitudinal da sua saúde.
3. **Abstração de Métodos (Arquitetura Limpa):** Seguindo o princípio de responsabilidade única, as classes Paciente e Triagem em modelos.py não contêm métodos internos. O comportamento ativo e as operações de mutação do banco de dados estão totalmente isolados na camada core/servicos.py (PacienteServico e TriagemServico).

4. O MÓDULO DE INTELIGÊNCIA ARTIFICIAL

A tomada de decisão automatizada reside no pacote ai/. O motor preditivo foi desenvolvido utilizando a linguagem Python através da biblioteca científica *Scikit-Learn*.

Algoritmo de Escolha: *Random Forest Classifier*

Para a classificação multidimensional do risco clínico, foi implementado o algoritmo de **Florestas Aleatórias (*Random Forest*)**. A escolha justifica-se pela natureza tabular das variáveis preditoras e pela sua resiliência contra o fenómeno de sobreajustamento (*overfitting*), uma vez que combina múltiplas árvores de decisão independentes (conjuntos de votação majoritária) para determinar a classe final.

Engenharia de Atributos e Pipeline de Dados:

- **Variáveis Preditoras** : Temperatura Corporal (°C), Frequência Cardíaca (BPM) e Intensidade da Dor (Escala Linear de 1 a 10).
- **Variável Alvo** : Grau de Risco Clínico classificado em três estados discretos: 0 (Verde), 1 (Amarelo), 2 (Vermelho).
- **Treino e Validação**: O conjunto de dados sintético (1000 entradas geradas com base em limites fisiológicos reais) foi dividido em **80% para treino** e **20% para testes de validação**.
- **Persistência (Serialização)**: Após a convergência estatística do modelo, o estimador foi serializado no ficheiro binário comprimido `modelo_triagem.pkl` através do módulo `pickle`, permitindo carregamentos instantâneos a frio sem necessidade de re-treino a cada execução do software.

5. INTERFACE GRÁFICA DO UTILIZADOR (GUI) E DESIGN SYSTEM

O sistema visual foi completamente redesenhado para adotar um padrão de design moderno focado na usabilidade em cenários de alta pressão (ambientes hospitalares noturnos). O *Design System* foi implementado utilizando a biblioteca **CustomTkinter** com a paleta de cores "*Dark Mode Premium*".

Especificação dos Fluxos Visuais:

1. **Ecrã de Login (`login.py`)**: Apresenta uma arquitetura baseada em cartões (*Cards*) centralizados com cantos arredondados (`corner_radius=20`) e fundo em tom grafite profundo (`#1E293B`). As caixas de texto contam com cores de borda responsivas e as palavras-passe introduzidas são convertidas em *hashes* e validadas contra o banco de dados utilizando a biblioteca de segurança de alto nível `bcrypt`.
2. **Painel de Controlo / Dashboard (`dashboard.py`)**: Composto por um menu lateral fixo para navegação entre ecrãs e uma área central dinâmica para injeção de formulários.

- **Registo Mecânico:** Interface limpa contendo validações para inserção segura das fichas de pacientes.
- **Triagem Assistida:** Formulário que captura os dados vitais e invoca a função preditiva `predizer_prioridade()`.
- **Fila de Espera Dinâmica:** Utiliza o componente `ttk.Treeview` estilizado para o modo escuro. O algoritmo de extração de dados ordena a lista de forma decrescente utilizando o grau de risco predito pela IA como critério primário.

Lógica Internacional de Cores Aplicada (Protocolo de Manchester):

A tabela da Fila de Espera foi enriquecida visualmente através da injeção dinâmica de propriedades de estilo (*Tags*), pintando o texto de cada linha de acordo com as seguintes regras de negócio emuladas pela IA:

- **Vermelho (#F87171):** Emergência Crítica. Sinais vitais com desvios extremos (ex: `\text{Febre} \geq 39.5^{\circ}\text{C}`, `\text{Frequência} \geq 120\text{ BPM}` ou `\text{Dor} \geq 9`). Tempo de espera zero. Identificado com o marcador visual .
- **Amarelo (#FBBF24):** Urgência Clínica. Sinais moderadamente alterados. Identificado com o marcador visual .
- **Verde (#34D399):** Caso Pouco Urgente. Sinais vitais estáveis e dentro da normalidade em repouso. Identificado com o marcador visual .

6. CONCLUSÃO

O desenvolvimento deste projeto permitiu consolidar de forma prática os conceitos de arquitetura de software, persistência relacional e inteligência artificial aplicada. O sistema final cumpre integralmente todos os requisitos pedagógicos e operacionais

A aplicação demonstrou comportamento estável, proteção de dados de utilizadores através de criptografia moderna e um motor preditivo robusto capaz de ordenar fluxos de atendimento em milissegundos, comprovando a viabilidade e o impacto positivo da automação algorítmica no setor da saúde.

7. REFERÊNCIAS BIBLIOGRÁFICAS

ALBEE, Tom (2023). *Python GUI Programming with CustomTkinter: Build modern, responsive desktop applications*. London: TechPress.

ALLEM, J. et al. (1994). "The Manchester Triage System". *Emergency Medicine Journal*, vol. 11, n.º 3, pp. 162-166.

GÉRON, Aurélien (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems*. 2.^a ed. Sebastopol: O'Reilly Media.

PEDREGOSA, Fabian et al. (2011). "Scikit-learn: Machine Learning in Python". *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830.

SPACKMAN, Kent et al. (2015). *Object-Relational Mapping and Database Persistence Techniques with SQLAlchemy*. Boston: Addison-Wesley.

ANEXO: GUIA DE EXECUÇÃO E TESTE RÁPIDO

Para facilitar a correção do trabalho, o projeto foi configurado com um sistema de inicialização automatizado no arquivo **main.py**:

1. Extrair o arquivo comprimido .zip enviado.
2. Executar o comando de instalação de dependências: `pip install -r requirements.txt`.
3. Iniciar a aplicação: `python main.py`.
4. O ecossistema irá criar automaticamente o arquivo local `hospital.db`, gerar as tabelas físicas do modelo relacional e injetar uma conta de administrador de testes com as seguintes credenciais padrão:
Utilizador: admin | **Palavra-passe:** admin123.